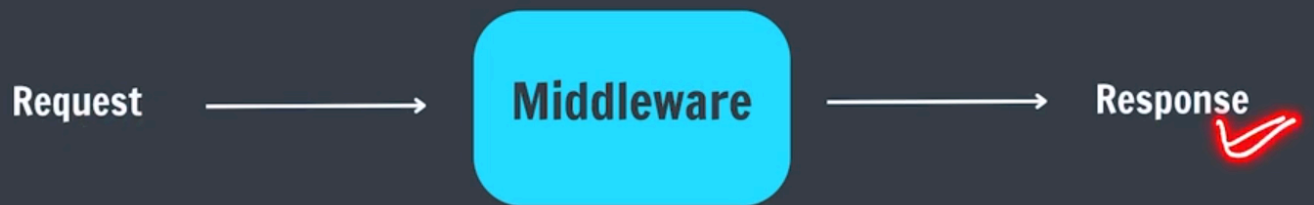


day 43

lec 1

Middleware

It is an intermediary



in Express

Middleware in Express are functions that come into play after the server receives the request and before the response is sent to the client.

body parser : request ki body ke andar ke data ko parse karna padta h , understandable form me convert karna padta h

express.static : backend ki static files ko frontend ko serve krne ke liye

express.urlencoded: andar ka data access kr paye

Middleware

Common middleware functions :

- `methodOverride` ✓
- `bodyParser`
- `express.static`
- `express.urlencoded`

```
app.use(express.urlencoded({ extended: true }));
```

```
app.use(express.static(path.join(__dirname, "/public")));
```

GET
POST
↓
PUT
DELETE

middleware properties :

1. can access req, res object
2. chaining
3. send a response (to break chaining , after sending response the later functions in chaining will not be called)
- 4.

lec 2

What do middlewares do?

Middleware functions can perform the following tasks:

- Execute any code.
- Make changes to the request and the response objects.
- End the request-response cycle.
- Call the next middleware function in the stack.

Our 1st Middleware

`app.use( middleware )`

```
app.use(() => {  
  console.log("hi, I am a middleware");  
});
```

using req & res object in middleware

```
app.use((req, res) => {  
  console.log("hi, I am a middleware");  
  res.send("bye");  
});
```



middleware can do two things , give response , or call next function or middleware

"

▼ MIDDLEWARES

> node_modules

JS app.js

{ } package-lock.json

{ } package.json

JS app.js >  app.use() callback

1 const express = require("express");

2 const app = express();

3

4 app.use((req, res) => {

5 | let { query } = req.query;

6 | console.log(query);

7 | console.log("Hi, I am middleware");

8 | res.send("middleware finished");

9 | });

10

11 app.get("/", (req, res) => {

12 | res.send("Hi, I am root.");

13 | });

14

15 app.get("/random", (req, res) => {

16 | res.send("this is a random page");

17 | });

18

19 app.listen(8080, () => {

20 | console.log("server listening to port 8080");

PROBLEMS

OUTPUT

TERMINAL

DEBUG CONSOLE

Hi, I am middleware

Hi, I am middleware

[nodemon] restarting due to changes...

[nodemon] starting `node app.js`

server listening to port 8080

[nodemon] restarting due to changes...

[nodemon] starting `node app.js`

server listening to port 8080

abcd

Hi, I am middleware



> OUTLINE

> TIMELINE

lec 3

Using next

The next middleware function is commonly denoted by a variable named **next**.

```
app.use((req, res, next) => {  
  console.log('Time:', Date.now())  
  next()  
})
```

If the current middleware function does not end the request-response cycle, it must call `next()` to pass control to the next middleware function.

you can go to the website to check the things

expressjs.com/en/guide/using-middleware.html

Using Express middleware x localhost:8080/random?query: x

expressjs.com/en/guide/using-middleware.html

Black Lives Matter.
Support the Equal Justice Initiative.

Express

search Home Getting started **Guide** API reference Advanced topics Resources

Middleware can also be declared in an array for reusability.

This example shows an array with a middleware sub-stack that handles GET requests to the `/user/:id` path

```
function logOriginalUrl (req, res, next) {
  console.log('Request URL:', req.originalUrl)
  next()
}

function logMethod (req, res, next) {
  console.log('Request Type:', req.method)
  next()
}

const logStuff = [logOriginalUrl, logMethod]
app.get('/user/:id', logStuff, (req, res, next) => {
  res.send('User Info')
})
```

Router-level middleware

Router-level middleware works in the same way as application-level middleware, except it is bound to an instance of `express.Router()`.

```
const router = express.Router()
```

Load router-level middleware by using the `router.use()` and `router.METHOD()` functions.

The following example code replicates the middleware system that is shown above for application-level middleware, by using router-level middleware:

```
const express = require('express')
const app = express()
```

go and see



Search Medium

Wr

5 Express Middleware Libraries Every Developer Should Know

Recommended Express middleware libraries



Bhagya Vithana · Follow

Published in Bits and Pieces · 7 min read · Dec 2, 2021

554

4



JS app.js > ...

```
1  const express = require("express");
2  const app = express();
3  
4  app.use((req, res, next) => {
5    console.log("Hi, I am 1st middleware");
6    next();
7  });
8
9  app.use((req, res, next) => {
10   console.log("Hi, I am 2nd middleware");
11   next();
12 });| 
13
14 app.get("/", (req, res) => {
15   res.send("Hi, I am root.");
16 });
17
18 app.get("/random", (req, res) => {
19   res.send("this is a random page");
20 });
```

PROBLEMS

OUTPUT

TERMINAL

DEBUG CONSOLE

```
server listening to port 8080
Hi, I am middleware
Hi, I am middleware
[nodemon] restarting due to changes...
[nodemon] starting `node app.js`
server listening to port 8080
Hi, I am 1st middleware
Hi, I am 2nd middleware
Hi, I am 1st middleware
Hi, I am 2nd middleware
█
```

even if you go to wrong path route , write something wrong in request body , still middleware will run

```
3
4  app.use((req, res, next) => {
5    console.log("Hi, I am 1st middleware");
6    next();
7    console.log("this is after next()");
8  });
9
10 app.use((req, res, next) => {
11   console.log("Hi, I am 2nd middleware");
12   next();
13 });
14
15 app.get("/", (req, res) => {
16   res.send("Hi, I am root.");
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
[nodemon] 3.0.1
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node app.js`
server listening to port 8080
Hi, I am 1st middleware
Hi, I am 2nd middleware
[nodemon] restarting due to changes...
[nodemon] starting `node app.js`
server listening to port 8080
Hi, I am 1st middleware
Hi, I am 2nd middleware
this is after next()
█
```

Ln 7, Col 20 (19 selected) Spaces: 4 UTF-8 L

next() ke baad kuchh likha to work krega

```

1  const app = express();
2
3
4  app.use((req, res, next) => {
5      console.log("Hi, I am 1st middleware");
6      return next();
7      console.log("abc");
8  });
9
10 app.use((req, res, next) => {
11     console.log("Hi, I am 2nd middleware");
12     next();
13 });
14
15 app.get("/", (req, res) => {
16     res.send("Hi, I am root.");

```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```

[nodemon] restarting due to changes...
[nodemon] starting `node app.js`
server listening to port 8080
Hi, I am 1st middleware
Hi, I am 2nd middleware
this is after next()
[nodemon] restarting due to changes...
[nodemon] starting `node app.js`
server listening to port 8080
[nodemon] restarting due to changes...
[nodemon] starting `node app.js`
server listening to port 8080
Hi, I am 1st middleware
Hi, I am 2nd middleware

```

Ln 7, Col 3 (19 selected) Spaces: 4 UTF-8

use return next so that a code after next doesn't get printed

lec 4

Creating Utility Middleware

Logger → log (useful info console print)

Requests
↓
'method' (Get, Post, Put, Del...)
time ✓
hostname

```
app.use((req, res, next) => {  
  req.responseTime = new Date(Date.now()).toString();  
  console.log(req.method, req.path, req.responseTime, req.hostname);  
  next();  
});
```

JS app.js > ...

```
11 // next();  
12 // });  
13  
14 app.get("/", (req, res) => {  
15   res.send("Hi, I am root.");  
16 });  
17  
18 app.get("/random", (req, res) => {  
19   res.send("this is a random page");  
20 });  
21  
22 app.use((req, res, next) => {  
23   req.time = Date.now();  
24   console.log(req.method, req.hostname, req.path, req.time);  
25   next();  
26 });  
27  
28 app.listen(8080, () => {
```

PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE

```
○ shradhakhapra@Shradhas-Air Middlewares % nodemon app.js  
[nodemon] 3.0.1  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting `node app.js`  
server listening to port 8080  
□
```

when we write the middleware after writing the route request response code , then the thing which will come first (route or middleware) will execute

lec 5

app.use Callback

```
14 app.use("/random", (req, res, next) => {  
15     console.log("I am only for random");  
16     next();  
17 })
```

```
27 //logger - morgan  
28 app.use((req, res, next) => {  
29     req.time = Date.now();  
30     console.log(req.method, req.hostname, req.path, req.time);  
31     next();  
32 });  
33  
34 //404  
35 app.use((req, res) => {  
36     res.status(404).send("Page not found!");  
37 });  
38  
39 app.listen(8080, () => {  
40     console.log("server listening to port 8080");  
41 });  
42
```

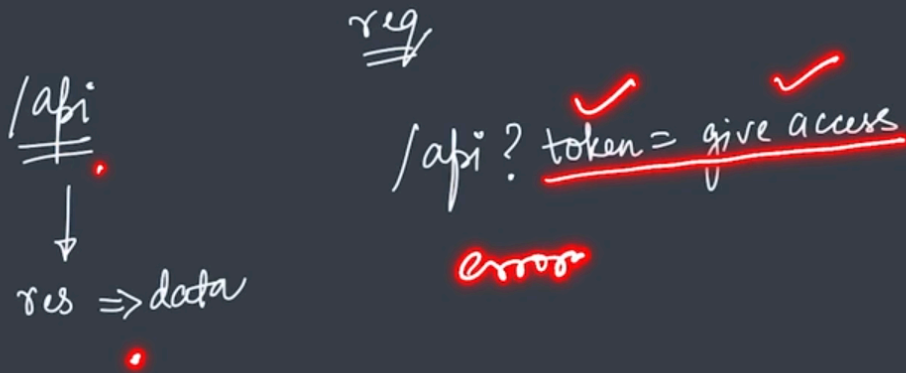
lect 6

API Token as Query String

Let's create a middle-ware for an api that checks if the access token was passed in the query string or not.

API Token as Query String

Let's create a middle-ware for an api that checks if the access token was passed in the query string or not.



```
13
14 app.use("/api", (req, res, next) => {
15   let { token } = req.query;
16   if (token === "giveaccess") {
17     next();
18   }
19   res.send("ACCESS DENIED!");
20 });
21
22 app.get("/api", (req, res) => {
23   res.send("data");
24 });
25
```

Multiple Middlewares

```
13
14 const checkToken = (req, res, next) => {
15   let { token } = req.query;
16   if (token === "giveaccess") {
17     next();
18   }
19   res.send("ACCESS DENIED!");
20 };
21 💡
22 app.get("/api", checkToken, (req, res) => {
23   res.send("data");
24 });
25
26 app.get("/", (req, res) => {
27   res.send("Hi, I am root.");
28 });
29
```

lec 8

Handling Errors

Express **Default** Error Handler

```
13
14 const checkToken = (req, res, next) => {
15   let { token } = req.query;
16   if (token === "giveaccess") {
17     next();
18   }
19   throw new Error("ACCESS DENIED!");
20 };
21
22 // app.get("/wrong", (req, res) => {
23 //   abcd = abcd;
24 // });
```